Module 04 — Exception Handling

Phase: Advanced Python
Previous module: OOP Part 4 — Abstract Classes & Interfaces
Next module: File Handling and Logging

Module Outcome:

- Type of errors — syntax vs runtime vs logical
- What an exception is, and why we handle it
- `try` / `except` — risky attempt and named recovery
- Catching multiple exceptions
- `else` and `finally` — success path vs cleanup
- `raise` and custom exception classes
- Common built-in exceptions

Three kinds of errors:

1. Syntax error — Python cannot even read the program. Missing `:`, `if pin = 1234` instead of `==`, a keyword used as a name. The file never really starts.

```
# Syntax error — Python refuses to parse this; try/except never runs
# if pin = 1234:
#     print("ok")
```

2. Runtime error (exception) — The program started, then hit a problem it cannot solve: `"abc"` to `int`, divide by zero, missing file. Python **raises an exception**. If nobody handles it, you get a traceback and the program **stops**. **This module is about these.**

```
int("abc")   # ValueError — runs, then fails
100 / 0      # ZeroDivisionError
```

3. Logical error — The program runs and finishes. The answer is just wrong: GST 18% coded as 8%, bill split among `friends` but you divide by `friends + 1`. No traceback. **Tests and careful reading** — not `except`.

```
price = 1000
gst = price * 0.08   # logical error: should be 0.18 — no exception, silent wrong total
```

Kind	When	Traceback?	`try` / `except` ?
Syntax error	Before the program runs	Yes (parse time)	No — edit the source
Runtime error / exception	While running	Yes if unhandled	Yes — this module
Logical error	Runs, wrong result	No	No — debug / test

Python's everyday word for a runtime failure is **exception**. Handling them — show a message, retry, shut down cleanly — is the rest of this module.

> **Note:**
> - `try` / `except` around a syntax error in the **same** file does not work; Python never reaches `try`.
> - A wrong business formula is not an exception until you **raise** one on purpose (`raise ValueError(...)`).

What is an exception?

A runtime error packaged as an **object** with a type (`ValueError`, `ZeroDivisionError`, `FileNotFoundError`) and a message. If nobody handles it, Python **stops** and prints a traceback.

Why handle exceptions?

Without handling, a wrong PIN or a missing CSV crashes the whole program. With handling, you show a clear message and continue (or shut down cleanly).

Think of an ATM: wrong PIN should not "explode" the machine — it shows "Invalid PIN" and lets the user retry.

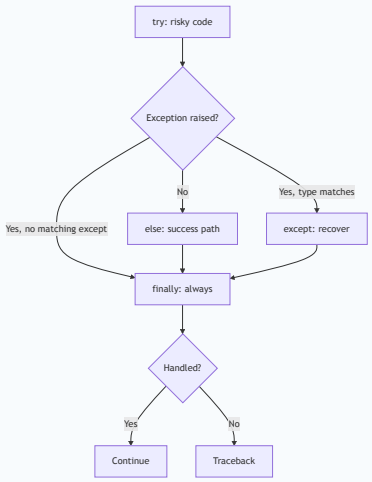
Python gives you **four keywords** for that story. Learn what each one is *for* before memorising the layout.

`try` — The risky attempt. Only the lines that might fail belong here: `int(pin)`, split a bill, look up a dict key. Keep it short. If `try` is 40 lines, you will not know which line raised.

`except` — Named recovery. "If this **type** of failure happened, do this instead of crashing." `except ValueError as e` binds the error object to `e` so you can print the message (`e`) or the type (`type(e).__name__`). You may stack several `except` blocks — Python uses the **first match**, so put specific types (`ValueError`) above broad ones (`Exception`). When two types share the same recovery, use a tuple: `except (TypeError, ValueError) as e:`.

`else` — The success path. Runs only when `try` finished with **no** exception. Put "PIN accepted — show the menu" here, not inside `try`. If that success code later bugs, `except` will not swallow it and pretend the PIN failed.

`finally` — Cleanup that **always** runs: success, caught error, even `return` from a function. ATM: return the card. UPI: log that an attempt happened. Do not put business decisions here (`if paid: ship_order()` belongs in `else`).



> **Key takeaways:**
> - `try` = attempt · `except` = recover by type · `else` = it worked · `finally` = cleanup
> - `else` is skipped when any exception was raised in `try`
> - `finally` still runs if you `return` inside `try` or `except`

Piece	Runs when	Typical job
`try`	Always entered first	Risky work only
`except`	Matching type raised in `try`	Message, retry, fallback

Piece	Runs when	Typical job
`else`	`try` succeeded (no exception)	Receipt, next screen
`finally`	Always, after the others	Return card, close, log

Remember: Bare `except:` also catches Ctrl+C — never use it. Catch specific types.

Custom exceptions name business failures clearly: `InsufficientBalanceError`, `SeatAlreadyBookedError` — better for readers and APIs than a bare `Exception`.

🏦 Bank 📱 UPI 📦 Order 🏥 Hospital 💰 Payroll

Where used?

- **SecureBank** / UPI — wrong PIN, insufficient balance, invalid amount
- Forms & CLI — bad age, non-numeric input, missing required fields
- File / CSV jobs — missing file, bad row converted with `int(...)`
- **TechMart** / **MedCare** — order capacity, appointment slot out of range
- Payments — domain errors (`PaymentError`) callers can catch separately

Important Concepts

1. try and except — risky attempt, named recovery

Definition: `try` runs code that might fail. `except Type` runs only when that type is raised — the program continues instead of dying with a traceback.

Purpose: User input and money maths fail often; crash-only programs are not shippable.

Put **only the risky lines** in `try`. Then name the doors:

```
raw_values = ["abc", "0", "4"]
for raw in raw_values:
    try:
        num = int(raw)      # may raise ValueError
        print(100 / num)    # may raise ZeroDivisionError
    except ValueError:
        print("Please enter a valid integer.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
```

Expected Output:

```
Please enter a valid integer.
Cannot divide by zero.
25.0
```

`except ValueError as e` binds the object so you can print a useful message. First matching `except` wins — list **specific** types before a broad `Exception`.

Note:

- Bare `except:` catches `KeyboardInterrupt` and hides bugs — catch specific exceptions.
- Swallowing exceptions with empty `except: pass` — log and handle, or re-raise.
- Using exceptions for flow control is slow and unreadable — use if/else for expected cases.

Common Error: Bare `except:` (or empty `except: pass`) hides real bugs and can block Ctrl+C. Catch specific types and log or re-raise.

Try these (2 programs):

Program 1 — SecureBank UPI PIN with retry

```
attempts = ["ab12", "1234"]

for attempt in attempts:
    try:
        pin = int(attempt)
        print("PIN accepted:", pin)
        break
    except ValueError as e:
        print("PIN must be digits only —", e)
```

Expected Output:

```
PIN must be digits only — invalid literal for int() with base 10: 'ab12'
PIN accepted: 1234
```

Program 2 — TechMart split bill safely

```
bill = 500
for friends in (4, 0):
    try:
        share = bill / friends
        print(f"Each pays ₹{share:.2f}")
    except ZeroDivisionError:
        print("Cannot split among 0 friends")
```

Expected Output:

```
Each pays ₹125.00
Cannot split among 0 friends
```

2. Catching multiple exceptions

Definition: One handler for several related error types, or several handlers when recovery differs.
Purpose: Same recovery path (e.g. “bad age data”) without duplicate blocks; different messages when the user should do different things.

Same recovery — one tuple:

```
try:
    age = int(data["age"])
except (KeyError, ValueError) as e:
    print("Bad age data:", e)
```

Different recovery — stack `except` blocks (first match wins):

```
try:
    age = int(data["age"])
except KeyError:
    print("age field is missing")
except ValueError:
    print("age must be a number")
```

Note:

- Over-broad `except Exception` masks real bugs — catch narrow types first.
- Wrong: `except ValueError, KeyError:` (old Python 2) — that is not a tuple. Use `except (ValueError, KeyError):`.

Try these:

Program 1 — AcmeHR marks: missing vs not a number

```
def load_marks(path, files):
    if path not in files:
        raise FileNotFoundError(path)
    return int(files[path])

files = {"marks.txt": "87", "bad.txt": "eighty"}
for path in ("marks.txt", "bad.txt", "missing.txt"):
    try:
        print("Marks:", load_marks(path, files))
    except (FileNotFoundError, ValueError) as e:
        print("Could not read marks:", type(e).__name__, "—", e)
```

Expected Output:

```
Marks: 87
Could not read marks: ValueError — invalid literal for int() with base 10: 'eighty'
Could not read marks: FileNotFoundError — missing.txt
```

3. else and finally — success path vs cleanup

Definition: `else` runs only when `try` succeeded. `finally` runs always — success, failure, or `return`.
Purpose: Keep "it worked" out of the error handler, and always finish cleanup (return the ATM card, log the attempt).

Why `else` exists

A common mistake is to put the success work *inside* `try`. Then a bug in the success work can be caught by `except` and reported as the original failure:

```
# Weak — a later bug looks like a bad PIN
try:
    pin = int("1234")
    print("Welcome")          # if this line fails, except may catch it
except ValueError:
    print("Invalid PIN")
```

`else` means: "`try` fully succeeded — now do the happy-path work." An error in `else` is **not** handled by this `except`.

```
try:
    pin = int("1234")
except ValueError:
    print("Invalid PIN")
else:
    print("Welcome")          # runs only if int() worked
```

Why `finally` exists

The card must come out whether the PIN was right or wrong. `finally` is that guarantee — including when a function **returns** from `try` or `except`.

Do **not** put business decisions in `finally` (`if paid: ship_order()`). That belongs in `else`. `finally` is cleanup and logging.

All four together:

```
try:
    amount = int("500")
    share = 1000 / amount
except ValueError:
    print("Need a number")
except ZeroDivisionError:
    print("Cannot use 0")
else:
    print(f"Each pays ₹{share:.2f}")
finally:
    print("Split attempt finished")
```

Expected Output:

```
Each pays ₹2.00
Split attempt finished
```

Note:

- `return` inside `finally` can suppress exceptions — keep `finally` for cleanup only.
- `else` does not run if `try` raised, even if `except` handled it.

Remember: `else` = try succeeded. `finally` = always runs (cleanup / close / log).

Interview: "`else` vs putting code in `try`" — `else` stops the success path from being swallowed by `except`.

Try these (2 programs):

Program 1 — SecureBank PIN: except, else, finally

```
CORRECT = 1234
for pin in ("ab12", "0000", "1234"):
    try:
        number = int(pin)
        if number != CORRECT:
```

```
            raise ValueError("Wrong PIN")
    except ValueError as e:
        print("Access denied:", e)
    else:
        print("Welcome to SecureBank")
    finally:
        print("Please remove your card")
    print("---")
```

Expected Output:

```
Access denied: invalid literal for int() with base 10: 'ab12'
Please remove your card
---
Access denied: Wrong PIN
Please remove your card
---
Welcome to SecureBank
Please remove your card
---
```

Program 2 — finally still runs after return

```
def demo():
    try:
        print("In try — about to return")
        return "success"
    finally:
        print("finally still runs before the return completes")

print("Result:", demo())
```

Expected Output:

```
In try — about to return
finally still runs before the return completes
Result: success
```

4. raise — intentional errors

Definition: You create an error on purpose when input or state is invalid.

Purpose: Fail fast with a clear message instead of continuing with bad data.

`raise ValueError("Amount must be positive")` stops the function and lets the **caller** decide how to recover (retry, show a form error, log).

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Not enough balance")
    return balance - amount

for amt in (200, -10, 5000):
    try:
        print("New balance:", withdraw(1000, amt))
```

```
    except ValueError as e:
        print("Withdraw failed:", e)
```

Expected Output:

```
New balance: 800
Withdraw failed: Amount must be positive
Withdraw failed: Not enough balance
```

Note:

- Raise a specific type (`ValueError`, or your custom class) — not a bare string (`raise "oops"` is invalid).
- After `raise`, the rest of that function does not run.

Try these:

Program 1 — MedCare book_seat capacity check

```
def book_seat(available, requested):
    if requested > available:
        raise ValueError("Not enough seats")
    print(f"Booked {requested} seat(s). Left: {available - requested}")

for avail, req in ((5, 2), (3, 5)):
    try:
        book_seat(avail, req)
    except ValueError as e:
        print("Booking failed:", e)
```

Expected Output:

```
Booked 2 seat(s). Left: 3
Booking failed: Not enough seats
```

5. Custom exception class

Definition: A class you write that inherits from `Exception`, named for a **business** failure.
Purpose: Callers can `except InsufficientBalanceError` without treating every `ValueError` the same — APIs and logs stay clear.

```
class InsufficientBalanceError(Exception):
    """Raised when withdrawal exceeds balance."""
    pass


class InvalidAccountError(Exception):
    def __init__(self, account_id, message="Invalid account"):
        self.account_id = account_id
        super().__init__(f"{message}: {account_id}")
```

Use it like any built-in: `raise InsufficientBalanceError("Not enough balance")`, then `except InsufficientBalanceError as e:`.

Note:

- Custom exception not inheriting `Exception` → not caught as expected — always inherit `Exception`.
- Empty `pass` is enough at first; add `__init__` when you need extra fields (`account_id`, `roll`).

Interview: Custom exceptions name domain failures (`UnderAgeError`, `InsufficientBalanceError`) so callers can catch them without treating every `Exception` the same.

Try these (2 programs):

Program 1 — InvalidRollError with roll stored

```
class InvalidRollError(Exception):
    def __init__(self, roll, message="Roll not found"):
        self.roll = roll
        super().__init__(f"{message}: {roll}")


def find_student(roll, db):
    if roll not in db:
        raise InvalidRollError(roll)
    return db[roll]


db = {101: "Anita", 102: "Ravi"}
for roll in (101, 999):
    try:
        print(find_student(roll, db))
    except InvalidRollError as e:
        print("Error:", e, "| roll attr:", e.roll)
```

Expected Output:

```
Anita
Error: Roll not found: 999 | roll attr: 999
```

Program 2 — Print, then re-raise

`raise` with no argument re-raises the **same** exception so the caller can still handle it.

```
def parse_marks(raw):
    try:
        return int(raw)
    except ValueError:
        print("ERROR: could not parse marks:", raw)
        raise


try:
    print("Marks:", parse_marks("eighty"))
except ValueError:
    print("Caller: show a form error, not a traceback")
```

Expected Output:

```
ERROR: could not parse marks: eighty
Caller: show a form error, not a traceback
```

6. Common built-in exceptions

Definition: Types Python already raises. You catch them by name.
Purpose: The type tells you *what* went wrong so the message can tell the user *what to do*.

Exception	Typical cause
`ValueError`	Wrong value (`int("abc")`)
`TypeError`	Wrong type operation (`"a" + 1`)
`IndexError`	Invalid list index
`KeyError`	Missing dict key
`FileNotFoundError`	File missing (you will use this a lot in File Handling)
`ZeroDivisionError`	`/` or `//` by zero
`AttributeError`	Missing attribute/method
`ImportError` / `ModuleNotFoundError`	Import failure

Try these:

Program 1 — Name that error quiz (print only the type name; `except Exception` is for this quiz only)

```
# Run each block — print only the type name. (except Exception is for this quiz only.)

try:
    "a" + 1
except Exception as e:
    print(type(e).__name__)   # TypeError

try:
    [1][5]
except Exception as e:
    print(type(e).__name__)   # IndexError

try:
    {}["missing"]
except Exception as e:
    print(type(e).__name__)   # KeyError

try:
    1 // 0
except Exception as e:
    print(type(e).__name__)   # ZeroDivisionError

try:
    int("hello")
except Exception as e:
    print(type(e).__name__)   # ValueError
```

Expected Output:

```
TypeError
IndexError
KeyError
ZeroDivisionError
ValueError
```

Step-by-Step: one assemble — SecureBank UPI

Type this with the class. Read it in order: `try` → `except` → `else` → `finally`. `finally` still runs when `except` or `else` **returns**.

```
def upi_transfer(pin, amount, balance):
    try:
        code = int(pin)
        if code != 4321:
            raise ValueError("Invalid PIN")
        if amount > balance:
            raise ValueError("Insufficient balance")
        new_balance = balance - amount
    except ValueError as e:
        print("Transfer failed:", e)
        return balance
    else:
        print(f"Sent ₹{amount}. New balance: ₹{new_balance}")
        return new_balance
    finally:
        print("Attempt logged")


print("Case 1 — bad PIN")
print("Returned:", upi_transfer("ab12", 200, 1000))
print()
print("Case 2 — success")
print("Returned:", upi_transfer("4321", 200, 1000))
print()
print("Case 3 — not enough money")
print("Returned:", upi_transfer("4321", 5000, 1000))
```

Expected Output:

```
Case 1 — bad PIN
Transfer failed: invalid literal for int() with base 10: 'ab12'
Attempt logged
Returned: 1000

Case 2 — success
Sent ₹200. New balance: ₹800
Attempt logged
Returned: 800

Case 3 — not enough money
Transfer failed: Insufficient balance
Attempt logged
Returned: 1000
```

Then open the kit (lab / tonight): `18_safe_input_exceptions` — finish `read_age` in `starter/safe_age.py`. `InvalidAgeError` is already there. Type it — do not paste `solution/`.

Practice Exercises

1. AcmeHR roll call: for each name, print the first letter. One name is `""` — catch `IndexError` so the loop continues.

Model answer:

```
names = ["Priya", "", "Ravi"]
for name in names:
    try:
        print(name, "→", name[0])
    except IndexError:
        print("(blank name — skipped)")
```

Expected Output:

```
Priya → P
(blank name — skipped)
Ravi → R
```

2. Parse a TechMart price written as `"₹1,200"`. Strip `₹` and commas, then `int`. Bad strings raise `ValueError` at the call site.

Model answer:

```
def parse_inr(raw):
    cleaned = raw.replace("₹", "").replace(",", "").strip()
    return int(cleaned)


for raw in ("₹1,200", "n/a"):
    try:
        print(raw, "→", parse_inr(raw))
```

```
    except ValueError:
        print(raw, "→ not a price")
```

3. TechMart stock: raise custom `OutOfStockError` when requested qty is greater than stock. Catch it at the call site (this is not the UPI PIN demo).

Model answer:

```
class OutOfStockError(Exception):
    pass


def take_from_shelf(stock, qty):
    if qty > stock:
        raise OutOfStockError(f"need {qty}, have {stock}")
    return stock - qty


for qty in (2, 10):
    try:
        print("Left:", take_from_shelf(5, qty))
    except OutOfStockError as e:
        print("Shelf:", e)
```

4. MedCare OTP: `try` `int(otp)`, `else` print accepted, `finally` print `"sms slot released"`. Use `"12ab"` and `"4321"` — not the SecureBank PIN story.

Model answer:

```
for otp in ("12ab", "4321"):
    try:
        code = int(otp)
        if code != 4321:
            raise ValueError("Wrong OTP")
    except ValueError as e:
        print("Denied:", e)
    else:
        print("OTP accepted")
    finally:
        print("sms slot released")
```

Mini-Project (Assignment): MedCare Safe Age Input CLI

Customer: MedCare registration desk. A clerk types an age. `"abc"` must not dump a traceback on the counter screen. `"0"` and `"200"` are not valid patients. `"21"` must go through.

Lab kit: 15_Lab_Assets/18_safe_input_exceptions/

File	What you do
`starter/safe_age.py`	Finish `read_age`. `InvalidAgeError` is already declared. Keep the `__main__` loop.
`broken/safe_age.py`	Bare `except:` returns `-1`; `"0"` looks successful. Fix both bugs.

File	What you do
`solution/`	Do **not** copy this. Use it only if you are stuck after you have tried.

This is **not** the SecureBank UPI assemble from class. Same tools (`try` / `raise` / custom exception), different ticket.

What to build

Piece	Detail
`InvalidAgeError`	Subclass of `Exception`. Raise when the number is not in **1–120**. Message includes the bad value (`age out of range: 0`).
`read_age(raw)`	`int(raw)` in `try`. Non-numeric → `ValueError` with message `age must be an integer` (re-raise or raise a new one — no traceback in `__main__`). In range → return the `int`.
Demo loop	Keep the four samples: `"21"`, `"abc"`, `"0"`, `"200"`. Print `sample`, then `->` and the age, **or** the exception type and message.
No type hints	Not required on this ticket. No `raise ... from e`. No `assert`.

Sample run

```
21 -> 21
abc ValueError age must be an integer
0 InvalidAgeError age out of range: 0
200 InvalidAgeError age out of range: 200
```

Stretch

- Wrap `read_age` in a `while True` prompt until a valid age is entered.
- Add `read_positive_float` for amounts (same pattern).
- **Later — Files week:** log failures with a timestamp (`datetime.now()`).

Done when

- [ ] `"21"` prints `21 -> 21`
- [ ] `"abc"` is caught as `ValueError` (`age must be an integer`) — no traceback
- [ ] `"0"` and `"200"` are `InvalidAgeError` (`age out of range: …`)
- [ ] `InvalidAgeError` inherits `Exception`
- [ ] `broken/` no longer uses bare `except:` and no longer accepts `0` / `200`
- [ ] Short README of sample outputs is pushed with the repo

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

Q1. try / except / else / finally execution order?
A: Enter `try`. If a matching `except` fires, run that handler and **skip** `else`. If `try` raised nothing, run `else`. `finally` always runs after that (including when `try` / `except` / `else` returns). If nothing matched, `finally` still runs, then the traceback continues.

Q2. Why `else` instead of putting success code in `try`?
A: Success code in `try` can raise and be caught by `except`, so you report the wrong failure. `else` runs only after `try` fully succeeded.

Q3. `except Exception` vs bare `except:`?
A: `except Exception` catches application errors but not `KeyboardInterrupt` / `SystemExit`. Bare `except:` catches those too — hides bugs and can block Ctrl+C. Catch specific types.

Q4. When create a custom exception?
A: When callers must distinguish a domain failure (**MedCare** `SlotNotAvailableError`, **SecureBank** `InsufficientBalanceError`) from a generic `ValueError`.

Q5. EAFP vs LBYL?
A: EAFP: try the operation, handle failure. LBYL: Look Before You Leap — check conditions first. Python culture favours EAFP for many I/O cases.

Q6. Exception vs `assert`?
A: `assert` is a developer sanity check (can be disabled with `-O`). Exceptions are for runtime errors and user-facing failures.

Q7. Why print/log before re-raising?
A: So the failure is recorded at the point it happened; `raise` then lets the caller decide the user-facing response. Catching and staying silent hides bugs.

Q8. Error vs exception in everyday language vs Python?
A: Everyday “error” covers syntax problems, runtime crashes, and wrong answers. In Python, **syntax errors** are parse-time (not for `try` / `except`), **exceptions** are runtime objects you can catch, and **logical errors** produce no traceback.

Q9. How would you handle errors in a SecureBank payment API?
A: Catch specific types (timeout vs invalid amount vs insufficient balance), map them to clear messages, log, and re-raise or return a safe result. Use a small hierarchy (`PaymentError` / `GatewayTimeoutError`) so callers can treat “any payment issue” or one subtype.
